📘 BPSC CS Module 01: Digital Electronics & Number Systems

📖 Master Classroom Edition & BPSC 5-Option Practice Module

> 🎯 **Mission Target:** 15 / 15 Marks in Part-III (BPSC TRE Computer Science)
> **Exam Philosophy:** Anti-GATE Rule (Direct, Conceptual, NCERT Class 11-12 & BCA Core Standard)
> **Golden BPSC Rule:** Rule of 2 — Har option ko scan karo. Do option sahi mile to Option (D) "More than one of the above" pakka!

📑 Table of Contents

1. **Chapter 1:** Number Systems & Fast Conversions (All Bases & Fractions)
2. **Chapter 2:** Binary Arithmetic, Complements & Overflow
3. **Chapter 3:** Binary Codes (BCD, Excess-3, Gray, Parity, Hamming Code)
4. **Chapter 4:** Boolean Algebra, Logic Laws & Canonical Forms (SOP & POS)
5. **Chapter 5:** Logic Gates (Basic, Universal, Exclusive & Bubble Equivalents)
6. **Chapter 6:** Karnaugh Maps (K-Map), Implicants & Minimization
7. **Chapter 7:** Combinational Logic Circuits (Adders, Subtractors, MUX, DEMUX, Decoders, Encoders)
8. **Chapter 8:** Sequential Logic Circuits (Latches, Flip-Flops, Tables & Race-Around)
9. **Chapter 9:** Registers & Counters (Shift Registers, Ring, Johnson, Ripple & Synchronous)
10. **Chapter 10:** BPSC TRE 5-Option Model Question Drill (With In-depth Trap Breakdown)

🏛️ CHAPTER 1: Number Systems & Fast Conversions

Computer hardware sirf electronic switches (Transistors) se bana hota hai, jo sirf do hi states samajhte hain: **ON (1)** ya **OFF (0)**. Isliye computer ka basic base binary (base 2) hota hai. Lekin human readability aur memory address representation ke liye hum Octal (Base 8) aur Hexadecimal (Base 16) use karte hain.

1.1 Base (Radix) & Digits Table

Number System	Base (Radix r)	Allowed Digits / Symbols	Ek Digit Kitne Bits Leta Hai?	Use Case
Binary	**2**	`0, 1`	1 bit	Computer hardware internal
Octal	**8**	`0, 1, 2, 3, 4, 5, 6, 7`	**3 bits** ($2^3 = 8$)	PDP minicomputers, UNIX permissions
Decimal	**10**	`0, 1, 2, 3, 4, 5, 6, 7, 8, 9`	~3.32 bits	Human calculation
Hexadecimal	**16**	`0-9` aur `A(10), B(11), C(12), D(13), E(14), F(15)`	**4 bits** ($2^4 = 16$)	Memory addresses, MAC address, Color codes

> 💡 **BPSC Trap:** Hexadecimal me letters case-insensitive hote hain (`A` ya `a` dono ka matlab 10 hai). `A` se lekar `F` tak values hamesha yaad honi chahiye:
> `A = 10`, `B = 11`, `C = 12`, `D = 13`, `E = 14`, `F = 15`.

1.2 Base Conversion Master Techniques

Method 1: Any Base r \to Decimal (Base 10)

Formula: Digits ko unki positional weight (r^i) se multiply karke jod do.

- Integer part: Point ke left side $r^0, r^1, r^2, \dots$
- Fractional part: Point ke right side $r^{-1}, r^{-2}, r^{-3}, \dots$

Example: $(1101.101)_2$ to Decimal

$$1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3}$$

$$= 8 + 4 + 0 + 1 + 0.5 + 0 + 0.125 = \mathbf{13.625_{10}}$$

Example: $(2A.4)_{16}$ to Decimal

$$2 \times 16^1 + 10 \times 16^0 + 4 \times 16^{-1} = 32 + 10 + 0.25 = \mathbf{42.25_{10}}$$

Method 2: Decimal (Base 10) \to Any Base r

- **Integer part ke liye:** Target base r se **Divide** karo, remainder ko **Bottom to Top (Neeche se Upar)** likho.
- **Fraction part ke liye:** Target base r se **Multiply** karo, integer part ko **Top to Bottom (Upar se Neeche)** likho.

```
Example: (25.625)₁₀ to Binary

Step A (Integer 25):                Step B (Fraction 0.625):
2 | 25                             0.625 × 2 = 1.25  ⟶ 1 bahar nikala (bacha 0.25)
2 | 12 - Remainder 1 (LSB)         0.25  × 2 = 0.50  ⟶ 0 bahar nikala (bacha 0.50)
2 |  6 - Remainder 0                0.50  × 2 = 1.00  ⟶ 1 bahar nikala (bacha 0.00 → FINISH!)
2 |  3 - Remainder 0
2 |  1 - Remainder 1                Upar se Neeche padho: .101
  |  0 - Remainder 1 (MSB)
Neeche se Upar padho: 11001

Final Result: (25.625)₁₀ = (11001.101)₂
```

Method 3: Binary \leftrightarrow Octal (3-Bit Grouping) & Hexadecimal (4-Bit Grouping)

Binary se Octal/Hex me badalne ke liye Decimal me jane ki koi zaroorat nahi hai. Direct grouping rule use karein:

```
                    [ DECIMAL POINT . ]
       INTEGER PART                          FRACTIONAL PART
  ⟸ (Right to Left)                       (Left to Right) ⟹
  Point se shuru karke PEECHE chalo       Point se shuru karke AAGE chalo
  (Kam pade to Left me 0 lagao)           (Kam pade to Right me 0 lagao)
```

- **Example (Binary to Octal):** $(1011010.11)_2$
- Integer part: `001` `011` `010` (Point ke bagal se 3-3 ka group banaya, aage zero lagaya) \to **1**, **3**, **2**
- Fraction part: `110` (Point ke bagal se 3 bit banane ke liye right me 0 lagaya) \to **6**
- **Result:** $(132.6)_8$
- **Example (Binary to Hexadecimal):** $(11010110.101)_2$
- Integer part: `1101` `0110` \to `13` (**D**), `6` \to **D6**
- Fraction part: `1010` (Right me 0 joda 4 bit poora karne ke liye) \to `10` (**A**)
- **Result:** $(D6.A)_{16}$

⚙️ CHAPTER 2: Binary Arithmetic, Complements & Overflow

2.1 r's Complement & (r-1)'s Complement

Kisi bhi base r me do tarah ke complement hote hain:

1. (r-1)**'s Complement (Radix Minus One):** Har digit ko base ke maximum digit se subtract karo.

- Binary me: **1's Complement** (Saari bits invert: `0 ↔ 1`).
- Decimal me: **9's Complement** (Har digit ko `9` se ghatao).

2. r**'s Complement (Radix Complement):**

$$\mathbf{r\text{'s Complement} = (r-1)\text{'s Complement} + 1}$$

- Binary me: **2's Complement** = $\text{1's Complement} + 1$.
- Decimal me: **10's Complement** = $\text{9's Complement} + 1$.

2.2 2's Complement 3-Second Inspection Shortcut

Exam me 1's complement nikal kar `+ 1` jodne me time waste mat karo:

```
[ 1   0   1   1   0   0 ]
  |   |   |   |   └───┘ ── Right side ke 0s: Bilkul SAME rakho (00)
  |   |   |   └─────────── Pehla 1 jo mila: Isko bhi SAME rakho (1)
  |   |   |
  └───┴───┴─────────────── Iske baad Left ki saari bits ko ULAT do (010)

  Result: 0 1 0 1 0 0
```

- **Niyam:** LSB (seedhe haath) se dekho. Jab tak pehla `1` na mile, saare 0 aur wo pehla `1` SAME copy karo. Uske baad left ki saari bits ko invert (`0 ko 1, 1 ko 0`) kar do!

2.3 Signed Number Representations & Range Comparison

Representation	MSB (Sign bit)	Positive Range	Negative Range	Zero (0) ke Representation	Total Distinct Numbers
Unsigned	Data bit	0 to +(2^n - 1)	None	1 (`00 ... 0`)	2^n
Signed Magnitude	0 = +, 1 = -	0 to +(2^{n-1} - 1)	-(2^{n-1} - 1) to 0	**2 (`+0` and `-0`)**	2^n - 1
1's Complement	0 = +, 1 = -	0 to +(2^{n-1} - 1)	-(2^{n-1} - 1) to 0	**2 (`+0` and `-0`)**	2^n - 1
2's Complement	0 = +, 1 = -	0 to +(2^{n-1} - 1)	-2^{n-1} **to** 0	**Sirf 1 (`00 ... 0`)**	2^n

> 🎯 **Direct Exam Fact for $n = 8$ Bits:**
> * 2's complement range: -128 **se** +127
> * 1's complement range: -127 **se** +127
> * 2's complement me 1 extra negative number hota hai: -128 **(`10000000`)** jiska koi positive counterpart 8-bit me nahi hota!
> * 2's complement me MSB ka weight negative hota hai: -2^{n-1} (8-bit me MSB = -128).

2.4 Overflow Condition in Signed Arithmetic

Jab do same sign ke numbers ko add karne par opposite sign ka result aa jaye, toh **Overflow** hota hai:

- $(+A) + (+B) \implies$ Agar result Negative aa jaye $\to$ **Overflow!**
- $(-A) + (-B) \implies$ Agar result Positive aa jaye $\to$ **Overflow!**
- Opposite sign ($+A$ aur $-B$) me overflow **kabhi nahi hota**.
- **Hardware Detection Formula:**

$$\mathbf{V = C_{\text{in}} \oplus C_{\text{out}}}$$

(MSB me aane wala Carry aur MSB se bahar nikalne wala Carry ka XOR agar `1` hai, toh Overflow hua hai!)

🏷️ CHAPTER 3: Binary Codes (BCD, Excess-3, Gray, Hamming)

3.1 Classification of Binary Codes

```
                    BINARY CODES
                         |
        ┌────────────────┴────────────────┐
Weighted Codes                          Non-Weighted Codes
(Har bit ka fixed weight)               (Position ka weight nahi hota)
- BCD (8421)                            - Excess-3 (XS-3)
- 2421, 5421                            - Gray Code (Unit Distance)
- 8 4 -2 -1
```

3.2 BCD Code (Binary Coded Decimal / 8421 Code)

- Decimal ke har ek digit (0 se 9) ko **4-bit binary** me represent kiya jata hai.
- Valid BCD numbers: `0000` (0) se lekar `1001` (9) tak.
- **Invalid BCD States:** 4 bit me total 16 combinations hote hain (0 to 15). BCD me sirf 10 valid hain (0 to 9).
- **Invalid States: 10 se 15** (`1010` se `1111`) $\to$ **Total 6 Invalid States** hoti hain!
- **BCD Addition Correction Rule:**
- Agar do BCD numbers ko jodne par sum > 9 aa jaye ya carry generate ho jaye, toh answer invalid hota hai.
- Correct karne ke liye answer me $+6$ **(** `0110` **)** add kiya jata hai!

3.3 Excess-3 Code (XS-3)

- BCD number me binary `3` (`0011`) jodne par Excess-3 code banta hai:

$$\mathbf{\text{Excess-3} = \text{BCD} + 0011_2}$$

- **Self-Complementing Code:** Excess-3 ka 1's complement lene par jo code milta hai, wo original decimal digit ka **9's complement** hota hai! (BPSC me bar-bar poocha jata hai: "Which of the following is a self-complementing code? $\implies$ **Excess-3 and 2421**").

3.4 Gray Code (Reflected Binary / Unit Distance Code)

- Gray code me do lagatar (consecutive) numbers ke beech me **sirf 1 bit** ka farak hota hai.
- Isliye ise **Unit Distance Code** ya **Minimum Error Code** kehte hain.
- Mechanical shaft encoders me rotary position measure karne ke liye use hota hai taaki transitions me glitch na aaye.

Fast Conversions:

- **Binary** $\to$ **Gray:** MSB same rakho ($G_3 = B_3$). Fir aage consecutive bits ka XOR karte jao ($G_2 = B_3 \oplus B_2$, $G_1 = B_2 \oplus B_1$).

- **Gray \to Binary:** MSB same rakho (B_3 = G_3). Fir result aur agle Gray bit ka diagonal XOR karo (B_2 = B_3 \oplus G_2, B_1 = B_2 \oplus G_1).

3.5 Error Detection & Correction: Hamming Code

- **Parity Bit:** Sirf 1-bit error detect kar sakta hai, correct nahi kar sakta.
- **Hamming Code:** Error detect bhi karta hai aur correct bhi karta hai.
- **Minimum Hamming Distance Formulas (d_{\min}):**

1. To **Detect** s bit errors: \mathbf{d_{\min} \ge s + 1}

2. To **Correct** t bit errors: \mathbf{d_{\min} \ge 2t + 1}

- **Parity Bits Calculation:** Agar data me m data bits hain aur p parity bits hain, toh formula:

$\mathbf{2^p \ge m + p + 1}$

- Parity bits hamesha 2 ki power positions par rakhi jati hain: Bit 1 (2^0), Bit 2 (2^1), Bit 4 (2^2), Bit 8 (2^3), etc.

📐 CHAPTER 4: Boolean Algebra & Canonical Forms

4.1 Boolean Algebra Laws & Postulates

Law Name	OR Form (+)	AND Form (\cdot)
Identity Law	A + 0 = A	A \cdot 1 = A
Null / Boundedness Law	A + 1 = 1	A \cdot 0 = 0
Idempotent Law	A + A = A	A \cdot A = A
Complement Law	A + A' = 1	A \cdot A' = 0
Commutative Law	A + B = B + A	A \cdot B = B \cdot A
Associative Law	(A + B) + C = A + (B + C)	(A \cdot B) \cdot C = A \cdot (B \cdot C)
Distributive Law	A + (B \cdot C) = (A + B)(A + C) *(Super Important!)*	A \cdot (B + C) = AB + AC
Absorption Law	A + AB = A aur A + A'B = A + B	A(A + B) = A aur A(A' + B) = AB
Consensus (Redundancy) Theorem	\mathbf{AB + A'C + BC = AB + A'C}	(A+B)(A'+C)(B+C) = (A+B)(A'+C)

> 💡 **Consensus Theorem Pehchanne ki Trick:**
> 1. Teen variables hone chahiye (A, B, C).
> 2. Har variable do baar aana chahiye.
> 3. Ek variable normal aur complemented form me ho (A aur A').
> \implies Jo doosra term bina complemented variable ka hai (BC), wo **Redundant (Faltu)** hota hai, use drop kar do!

4.2 De Morgan's Laws

1. \mathbf{(A + B)' = A' \cdot B'} (NAND equivalent to Bubbled OR)

2. \mathbf{(A \cdot B)' = A' + B'} (NOR equivalent to Bubbled AND)

- Rule: "Break the line, change the sign" (Upar ki bar todo, aur beech ka sign badlo: `+` ko `.` aur `.` ko `+`).

4.3 Duality Principle

Kisi bhi valid Boolean equation ka **Dual** nikalne ke liye:

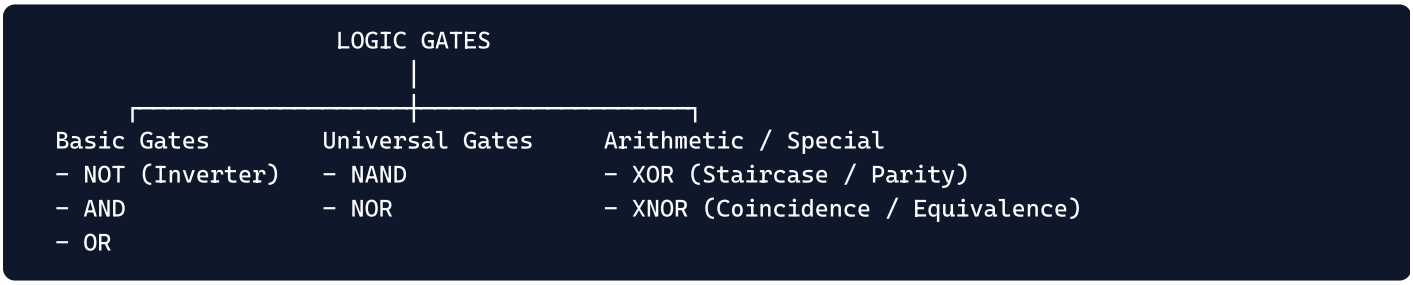
- `+` (OR) ko badlo `.` (AND) me.
- `.` (AND) ko badlo `+` (OR) me.
- `0` ko badlo `1` me, aur `1` ko badlo `0` me.
- *(Variables ko complement nahi kiya jata!)*

4.4 Canonical Forms: Minterms vs Maxterms

Feature	**Minterms (m)**	**Maxterms (M)**
Expression Form	**SOP (Sum of Products)** e.g., $A'B + AB'$	**POS (Product of Sums)** e.g., $(A+B)(A'+B')$
Output Value	Function ki value **1** hone par banta hai	Function ki value **0** hone par banta hai
Variable Rule	$1 \implies A$, $0 \implies A'$	$0 \implies A$, $1 \implies A'$
Symbol	$\Sigma m(1, 2, \dots)$	$\Pi M(0, 3, \dots)$
Relationship	$\mathbf{m_i' = M_i}$ (Minterm ka complement = Maxterm)	$\mathbf{M_i' = m_i}$

🔌 CHAPTER 5: Logic Gates & Universal Realization

5.1 Basic, Universal & Special Gates Summary



1. XOR Gate ($\oplus$):

- Output $Y = A \oplus B = A'B + AB'$
- **Odd 1s Detector:** Jab inputs me `1` ki sankhya odd hoti hai, tab output `1` hota hai.
- **Controlled Inverter:**
- $A \oplus 0 = A$ (Buffer / Pass)
- $A \oplus 1 = A'$ (Inverter)
- **Properties:** Associative aur Commutative dono hota hai ($A \oplus A = 0$, $A \oplus 0 = A$).

2. XNOR Gate ($\odot$):

- Output $Y = A \odot B = AB + A'B' = (A \oplus B)'$
- **Even 1s Detector / Coincidence Gate:** Dono input same hone par output `1` deta hai.
- For odd number of variables: XOR aur XNOR functions equal hote hain. For even number of variables: $A \odot B = (A \oplus B)'$.

5.2 Universal Gates Realization Table (Memory Trick: 1, 2, 3, 4, 5)

Dono universal gates ke liye bas ye sequence dhyan me rakhein:

```
NAND Gates Sequence:  NOT (1) ──> AND (2) ──> OR (3) ──> XOR (4) ──> XNOR (5)
NOR Gates Sequence:   NOT (1) ──> OR (2)  ──> AND (3) ──> XNOR (4) ──> XOR (5)
```

Target Gate	Minimum NAND Gates	Minimum NOR Gates
NOT	**1**	**1**
AND	**2**	**3**
OR	**3**	**2**
NOR	**4**	**1**
NAND	**1**	**4**
XOR	**4**	**5**
XNOR	**5**	**4**

🗺️ CHAPTER 6: Karnaugh Maps (K-Map) & Minimization

6.1 K-Map Structure & Gray Code Ordering

K-Map me adjacent cells ke beech sirf 1 bit ka difference hona chahiye, isliye cells ki numbering hamesha **Gray Code order** me hoti hai: `00, 01, 11, 10` (BPSC trap: `11` pehle aata hai `10` se!).

- n variables ke K-Map me total 2^n **cells** hote hain:
- 2-variable: 4 cells
- 3-variable: 8 cells
- 4-variable: 16 cells

```
          3-Variable K-Map (A \ BC)
            00     01     11     10
        ┌──────┬──────┬──────┬──────┐
   0    │  m0  │  m1  │  m3  │  m2  │
        ├──────┼──────┼──────┼──────┤
   1    │  m4  │  m5  │  m7  │  m6  │
        └──────┴──────┴──────┴──────┘
```

6.2 Grouping Rules & Definitions

1. **Allowed Group Sizes:** Sirf 2 ki powers me group bante hain ($1, 2, 4, 8, 16$). K-Map me 3 ya 6 ka group banana **paap** hai!

2. **Variable Elimination Rule:**

- 2 ka group (Pair) $\implies$ **1 variable eliminate** hota hai.
- 4 ka group (Quad) $\implies$ **2 variables eliminate** hote hain.
- 8 ka group (Octet) $\implies$ **3 variables eliminate** hote hain.

3. **Important Terms:**

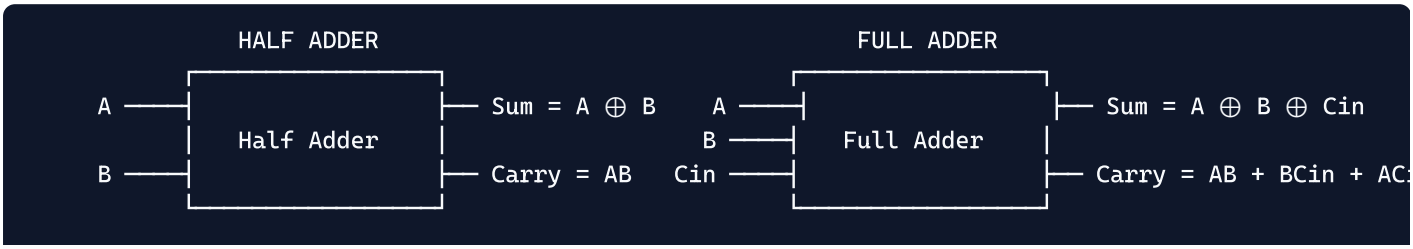
- **Implicant:** K-Map ka koi bhi single `1`.
- **Prime Implicant (PI):** Sabse bada group jo kisi doosre group me poori tarah na samaye.
- **Essential Prime Implicant (EPI):** Aisa Prime Implicant jisme kam se kam ek aisa `1` ho jo kisi aur group me shamil nahi ho sakta. Yeh final simplified answer me **hamesha aayega**.

- **Don't Care Condition (X ya d):** Aisi input combination jo system me kabhi occur nahi hogi. Isko group bada karne ke liye `1` ki tarah treat kar sakte hain, lekin alag se iska group banana zaroori nahi hai.

CHAPTER 7: Combinational Logic Circuits

Combinational circuits me koi memory nahi hoti; inka output **sirf present input par depend karta hai** (No feedback, no clock).

7.1 Half Adder & Full Adder



Parameter	**Half Adder**	**Full Adder**
Inputs	2 inputs (A, B)	3 inputs (A, B, C_{\text{in}})
Outputs	2 outputs (Sum, Carry)	2 outputs (Sum, Carry)
Sum Equation	S = A \oplus B	S = A \oplus B \oplus C_{\text{in}}
Carry Equation	C = AB	C_{\text{out}} = AB + BC_{\text{in}} + AC_{\text{in}} = AB + C_{\text{in}}(A \oplus B)
Hardware Required	**1 XOR + 1 AND**	**2 Half Adders + 1 OR Gate**
Minimum NANDs	**5 NAND Gates**	**9 NAND Gates**

7.2 Half Subtractor & Full Subtractor

- **Half Subtractor:** \text{Difference} = A \oplus B, \text{Borrow} = A'B (Hardware: 1 XOR + 1 AND with inverted input).
- **Full Subtractor:** \text{Difference} = A \oplus B \oplus B_{\text{in}}, \text{Borrow} = A'B + A'B_{\text{in}} + BB_{\text{in}}.

7.3 Multiplexer (MUX) — Data Selector

- MUX ek aisi circuit hai jisme 2^n input lines hoti hain, n select lines hoti hain, aur **sirf 1 output** hota hai (2^n : 1).
- Formula: \text{Inputs} = 2^{\text{Select Lines}}
- MUX ko **Universal Logic Circuit** kaha jata hai kyunki kisi bhi Boolean function ko bina extra gates ke MUX se implement kiya ja sakta hai.
- **Tree Multiplexer Shortcut Formula:**

Chhote 2 \times 1 MUX se bada N \times 1 MUX banane ke liye hamesha N - 1 MUX lagte hain!

- 4 \times 1 ke liye: 4 - 1 = \mathbf{3}
- 8 \times 1 ke liye: 8 - 1 = \mathbf{7}
- 16 \times 1 ke liye: 16 - 1 = \mathbf{15}
- 64 \times 1 ke liye: 64 - 1 = \mathbf{63}

7.4 Demultiplexer & Decoders

- **Demultiplexer (DEMUX):** 1 input ko 2^n output me distribute karta hai using n select lines ($1 : 2^n$). MUX ka theek ulta.
- **Decoder ($n : 2^n$):** n input lines ko 2^n output lines me decode karta hai.
- $2 : 4$ Decoder, $3 : 8$ Decoder (Octal decoder), $4 : 16$ Decoder.
- Decoder + External OR Gate lagakar koi bhi Boolean function implement kiya ja sakta hai.
- **Encoder ($2^n : n$):** Decoder ka theek ulta. Active input ko binary code me convert karta hai.
- **Priority Encoder:** Agar ek se zyada input ek sath active ho jayein, toh highest priority wale input ko output code banata hai.

🔄 CHAPTER 8: Sequential Logic Circuits (Flip-Flops)

Sequential circuits me **Memory element (Feedback)** hota hai. Output **present input aur past output dono par depend karta hai**.

8.1 Latch vs. Flip-Flop

Feature	**Latch**	**Flip-Flop**
Clock Signal	Clock nahi hoti (Asynchronous)	Clock pulse hoti hai (Synchronous)
Triggering	**Level-Triggered** (High ya Low voltage par sensitive)	**Edge-Triggered** (Positive $\uparrow$ ya Negative $\downarrow$ edge par sensitive)
Speed	Tez hota hai	Thoda slow hota hai
Reliability	Glitches/Noise aane par output badal sakta hai	Reliable aur predictable hota hai

8.2 Master Flip-Flop Comparison Table

Flip-Flop	Inputs	Characteristic Equation (Next State)	Excitation Table ($Q_n \to Q_{n+1}$)	Key BPSC Exam Feature
SR	S, R	$\mathbf{Q_{n+1} = S + R'Q_n}$ ($SR = 0$)	$0\to0$: S=0, R=X $0\to1$: S=1, R=0 $1\to0$: S=0, R=1 $1\to1$: S=X, R=0	$S=1, R=1$ is **Invalid / Forbidden**
JK	J, K	$\mathbf{Q_{n+1} = JQ_n' + K'Q_n}$	$0\to0$: J=0, K=X $0\to1$: J=1, K=X $1\to0$: J=X, K=1 $1\to1$: J=X, K=0	$J=1, K=1$ par **Toggle** karta hai (Race-around trap)
D (Data)	D	$\mathbf{Q_{n+1} = D}$	$0\to0$: D=0 $0\to1$: D=1 $1\to0$: D=0 $1\to1$: D=1	**Delay / Buffer / Transparent latch**
T (Toggle)	T	$\mathbf{Q_{n+1} = T \oplus Q_n}$	$0\to0$: T=0 $0\to1$: T=1 $1\to0$: T=1 $1\to1$: T=0	$T=1 \implies$ Toggle; **Frequency Divider ($\div 2$)**

8.3 Race-Around Condition in JK Flip-Flop

- **Kya hota hai?** Jab $J = 1$ aur $K = 1$ ho, aur clock pulse width (t_p) flip-flop ke propagation delay (t_{pd}) se badi ho ($t_p > t_{pd}$), toh ek hi clock pulse ke dauran output lagatar $0 \to 1 \to 0 \to 1$ toggle karta rehta hai. Clock khatam hone par state unpredictable ho jati hai.
- **Iske teen ilaj (Solutions):**

1. **Master-Slave JK Flip-Flop** use karo (Master positive edge par kaam karega, Slave negative edge par).
2. **Edge Triggering** use karo (Narrow spike triggers).
3. Clock pulse width ko propagation delay se kam rakho ($t_p < t_{pd}$).

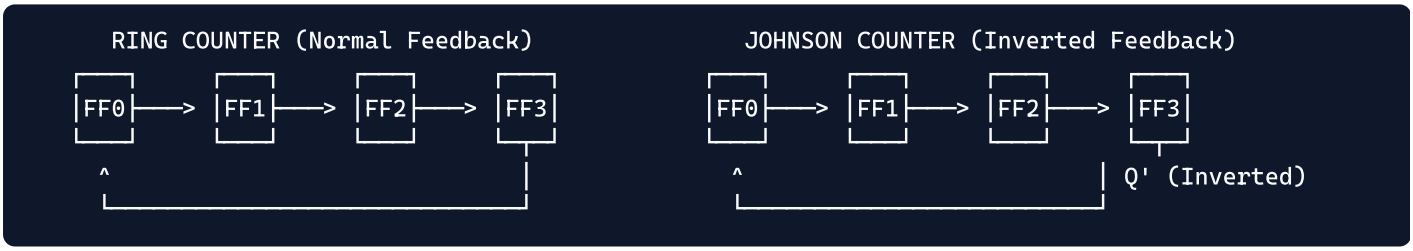
🔢 CHAPTER 9: Registers & Counters

9.1 Shift Registers

n-bit shift register banane ke liye n **Flip-Flops** ki zaroorat hoti hai.

1. **SISO (Serial In Serial Out):** n bits enter karne me n clock cycles, aur bahar nikalne me $n-1$ clock cycles. Total = $2n - 1$.
2. **SIPO (Serial In Parallel Out):** Data enter karne me n clock cycles, output 1 cycle me parallel mil jata hai.
3. **PISO (Parallel In Serial Out):** Data load 1 cycle me, serial out me $n-1$ cycles.
4. **PIPO (Parallel In Parallel Out):** Sabse fast! Read aur Write dono **1 clock cycle** me ho jata hai.

9.2 Ring Counter vs Johnson Counter



Parameter	**Ring Counter**	**Johnson Counter (Twisted Ring / Moebius)**
Feedback Connection	Last flip-flop ka normal output (Q) pehle FF me	Last flip-flop ka inverted output (Q') pehle FF me
Total States (Modulus)	N **States** (MOD-N)	$2N$ **States** (MOD-$2N$)
Unused States	$2^N - N$	$2^N - 2N$
Decoding Circuit	Kisi decoder ki zaroorat nahi hoti	2-input AND/OR gates chahiye
4-bit Flip-Flop Example	States = **4**, Unused = $16 - 4 =$ **12**	States = **8**, Unused = $16 - 8 =$ **8**

9.3 Asynchronous (Ripple) vs Synchronous Counters

- **Asynchronous (Ripple) Counter:**
- Pehle Flip-Flop ko external clock di jati hai. Agle Flip-Flop ki clock pichhle Flip-Flop ka output (Q ya Q') hota hai.
- **Ripple Delay:** Cumulative propagation delay $T = n \times t_{pd}$. Slow hota hai.
- **Synchronous Counter:**
- Saare Flip-Flops ko **ek sath same common clock** di jati hai.
- Bohat tez (Fast) hota hai kyunki koi ripple delay nahi hota.
- **Flip-Flops Required Formula:** Agar counter ko N states count karni hain (MOD-N counter):

$$\mathbf{2^n \ge N} \implies n = \lceil \log_2 N \rceil$$

- Example: MOD-10 (Decade counter) ke liye $2^n \ge 10 \implies n = 4$ **Flip-Flops** lagenge.

🎯 CHAPTER 10: BPSC TRE 5-Option Model Drill

Question 1: Number Systems

The octal equivalent of the hexadecimal number $(A7.C)_{16}$ is:

(A) $(247.6)_8$

(B) $(247.3)_8$

(C) $(517.6)_8$

(D) More than one of the above

(E) None of the above

> **Correct Answer: (A)**
> **Post-Mortem & Method:**
> Step 1: Hexadecimal ko 4-bit binary me todo:
> $A = 1010$, $7 = 0111$, $. = .$, $C = 1100 \implies (10100111.1100)_2$
> Step 2: Binary ko 3-3 bit group me badlo:
> Point ke pehle (←): 010 (2), 100 (4), 111 (7) $\implies$ **247**
> Point ke baad (→): 110 (6) $\implies$ **.6**
> Final Result = $(247.6)_8$. Option (A) is strictly correct.

Question 2: Complements & Ranges

In an 8-bit 2's complement representation, which of the following statements is/are correct?

(A) The range of representable integers is -128 to $+127$.

(B) The number zero has a unique representation (00000000).

(C) The binary sequence 10000000 represents decimal -128.

(D) More than one of the above

(E) None of the above

> **Correct Answer: (D) More than one of the above**
> **Rule of 2 Trap:**
> Option (A) is correct (-2^7 to $+2^7 - 1 = -128$ to $+127$).
> Option (B) is correct (Unlike 1's complement, 2's complement has only one zero).
> Do option sahi milte hi **Rule of 2** trigger hota hai $\implies$ Answer is guaranteed **(D)**! (Option C is also correct).

Question 3: Codes

Which of the following binary codes is/are classified as non-weighted code(s)?

(A) Excess-3 Code

(B) Gray Code

(C) BCD (8421) Code

(D) More than one of the above

(E) None of the above

> **Correct Answer: (D) More than one of the above**
> **Post-Mortem:**
> Excess-3 and Gray code dono me positions ka koi mathematical weight $(8, 4, 2, 1)$ nahi hota. Isliye dono non-weighted hain. Do option sahi $\implies$ Option (D)!

Question 4: Universal Gates

How many minimum 2-input NAND gates are required to implement a 2-input XOR gate and XNOR gate respectively?

(A) 4 and 5

(B) 5 and 4

(C) 4 and 4

(D) More than one of the above

(E) None of the above

> **Correct Answer: (A)**
> **Sequence Memory Trick:**
> NAND Sequence: `NOT(1) → AND(2) → OR(3) → XOR(4) → XNOR(5)`.
> Isliye XOR = 4 NAND gates, aur XNOR = 5 NAND gates. Option (A) is exact.

Question 5: Sequential Circuits

A 4-bit Ring counter and a 4-bit Johnson counter have how many total distinct states respectively?

(A) 4 and 8

(B) 16 and 8

(C) 4 and 16

(D) More than one of the above

(E) None of the above

> **Correct Answer: (A)**
> **Post-Mortem:**
> For N flip-flops:
> Ring Counter states = $N = 4$.
> Johnson Counter states = $2N = 2 \times 4 = 8$.
> Isliye 4 and 8. Option (A) is correct.